



**PARIS
LODRON
UNIVERSITÄT
SALZBURG**

FINAL PROJECT REPORT

DATABASE BACKEND FOR WEBGIS CHRISTMAS MARKET IN GDAŃSK, POLAND

Submitted by :

**Rabina Twayana
I23I3887**

Submitted to :

Dr. Martin Sudman



TABLE OF CONTENTS

1	INTRODUCTION	1
2	STUDY AREA	2
3	DATA PREPARATION.....	3
3.1	Georeferencing.....	3
3.2	Digitization and data population.....	4
4	IMPLEMENTATION	5
4.1	Create Database and Create PostGIS Extension.....	5
4.2	Connect Database in QGIS.....	5
4.3	Load Shapefiles into Database.....	6
4.4	Build Entity-Relation Diagram using ERD Tool in Pgadmin4	7
4.5	Create Tables in Database	9
4.6	Insert Data into Database and Database Normalization.....	12
4.7	Create View	17
4.8	Define Roles and Permissions	18
5	QUERIES	23
5.1	Queries based on User's Location	23
5.2	Queries not based on User's Location.....	25
6	PGROUTING IMPLEMENTATION	27
6.1	Installation and create extension.....	27
6.2	Building Routing Topology.....	27
6.3	Routing Queries	32
7	EXPORT DDL AND DML	35
8	CONCLUSION	35
	REFERENCES	35

1 INTRODUCTION

A web-based platform or mobile application that integrates GIS capabilities offers a unique opportunity for event organizers to provide interactive and dynamic spatial services to users. For this database, the backend plays an important role in efficient spatial data management and queries.

The main objective of the project is to develop a spatial database that serves as a backend for a city festival to fulfill the following requirements:

- Organise the spaces/stalls/attractions as well as other facilities.
- User-specific, dynamic queries that return which events take place or facilities that are close to the current visitor's position.
- User-specific and dynamic queries about the festival, such as digital maps, lists of events etc.

For the case study, the Christmas fair in Gdańsk was selected. It is a popular Christmas town that attracts crowds of interested people every year. Visitors can buy Christmas decorations, original gifts, Christmas delicacies, play games, enjoy events and performances, and try cuisines from all over the world.

2 STUDY AREA



Figure 1: Christmas Market in Gdansk, Source: (The magic of Christmas in the center of Gdansk. Crowds at the Christmas Market, 2024)



Figure 2: Map of the Christmas Market, Gdansk, Source: (The magic of Christmas in the center of Gdansk. Crowds at the Christmas Market, 2024)

3 DATA PREPARATION

3.1 Georeferencing

Using the [georeferencer tool](#) in QGIS, map of the christmas market is georeferenced with the OSM base map. Four ground control points were selected and added to the tool. Figure 3 represents the georeferencer tool with the selected GCP (Ground Control Points) enclosed with a purple box. Figure 4 is the result of georeferencing.

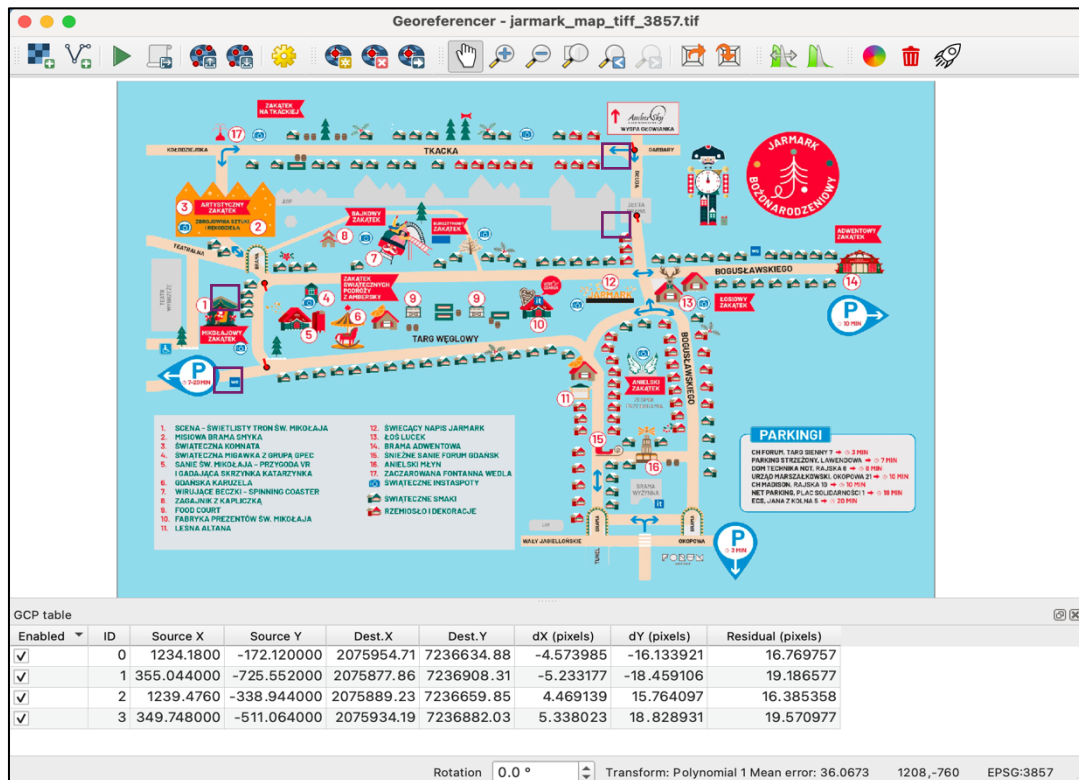


Figure 3: Georeferencing

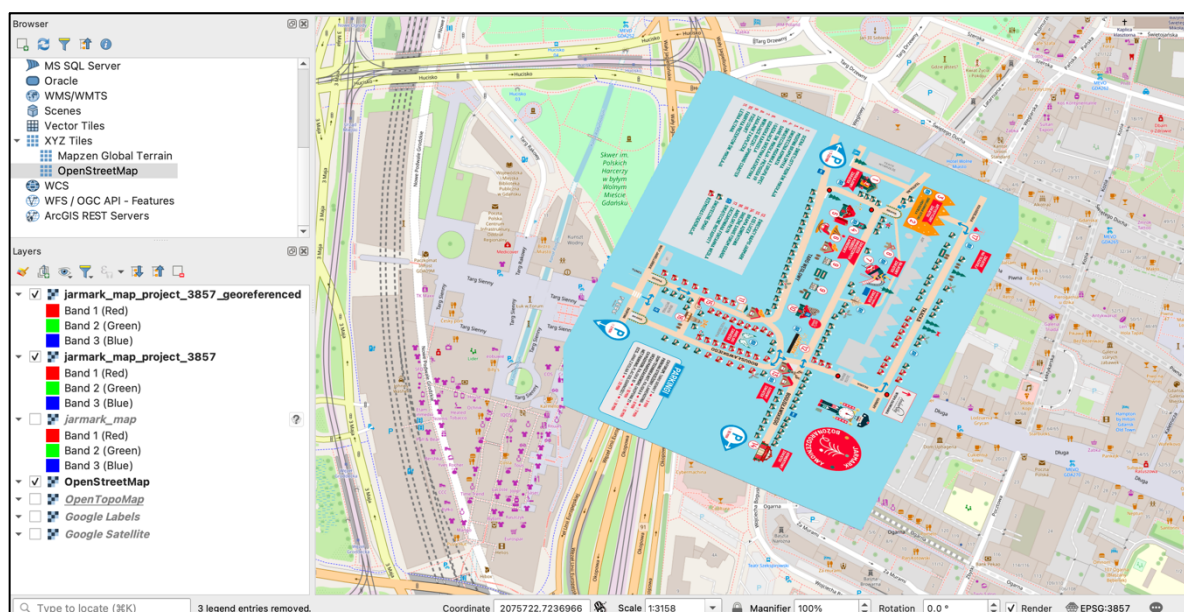


Figure 4: Georeferenced Christmas Market Map

3.2 Digitization and data population

Three new shapefile layers were created to digitise and store information related to facility, pathway, and attractions of point, linestring, and polygon geometry type, respectively, in the EPSG:3857 coordinate system. Figure 5 represents the digitized map and Figures 6 (a), (b), and (c) represent the attribute table of pathway, facility, and attraction layer, respectively.

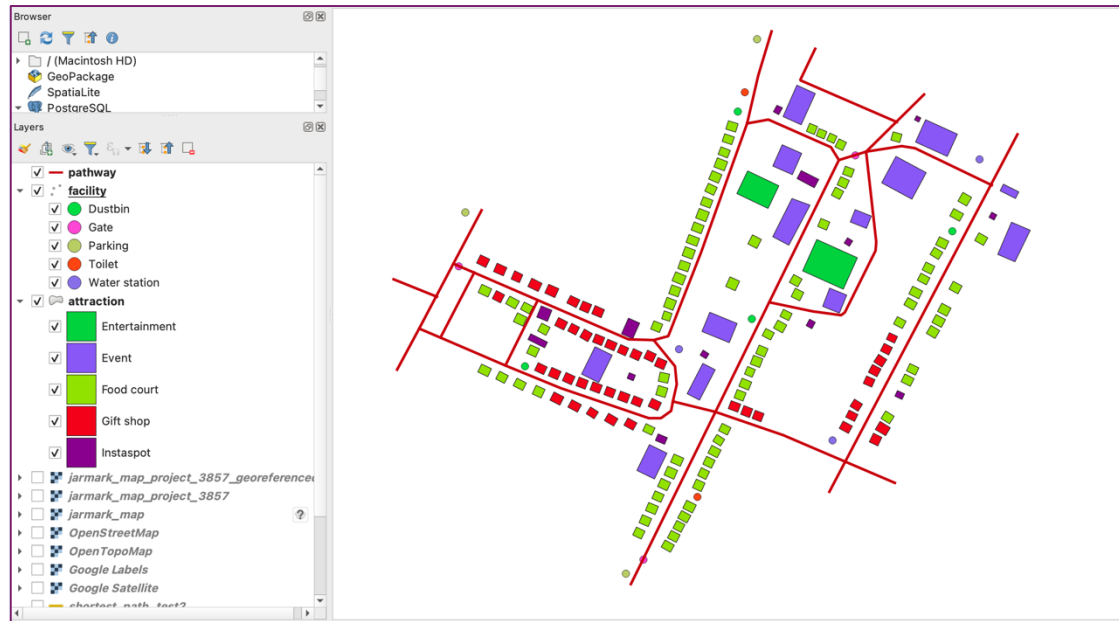


Figure 5: Digitized Map

a

pathway — Features Total: 15, Filtered: 15, Selected: 0

id	name	code
1	1 Tkacka	1
2	2 Długa	2
3	3 Targ Weglowy	3
4	4 Waly Jagiello...	4
5	5 Waly Jagiello...	5
6	6 Targ Sienny	6
7	7 Targ Weglowy	7

b

facility — Features Total: 15, Filtered: 15, Selected: 0

id	name	type	code
1	1 P1	Parking	1
2	2 WC1	Toilet	2
3	3 WC2	Toilet	3
4	4 P2	Parking	4
5	5 P3	Parking	5
6	6 Advent Gate	Gate	6
7	7 DB4	Dustbin	7

c

attraction — Features Total: 159, Filtered: 159, Selected: 0

id	name	type	code	ven_name	ven_adr	ven_con
13	13 Food court 1	Food court	13	Justin House	747 Pugh Bur...	+43 647862...
14	14 Food court 2	Food court	14	Teresa Cochran	69201 Steve...	+43 650744...
15	15 Lightening Sign	Event	15	Adam Williams	102 James P...	+43 684663...
16	16 Food court 3	Food court	16	Dr.John Petersen	8914 Leonar...	+43 623342...
17	17 Scene - The Glorius Thro...	Event	17	Nicholas Chase	581 Jennifer ...	+43 620454...
18	18 Snapshot with GPEC Group	Event	18	David King	3875 Mullins ...	+43 6901841...
19	19 Food court 4	Food court	19	Joanna Rodriguez	16545 Kevin ...	+43 698921...
20	20 Grove with a Chapel	Event	20	Mark Yang	9518 Young ...	+43 6474156...
21	21 Instaspot 7	Instaspot	21	Andrew Carr	51245 Jessic...	+43 6758317...
22	22 Instaspot 9	Instaspot	22	Kathryn Anderson	USNS Hodge,...	+43 638436...

Figure 6: Attribute Tables, [a] Pathway [b] Facility [c] Attraction

4 IMPLEMENTATION

4.1 Create Database and Create PostGIS Extension

New database was created by running following SQL command in the default postgres query tool.

```
CREATE DATABASE city_festival_management;
```

Then, the respective query tool was opened for the newly created database and the command shown in Figure 7 was runned for creating the PostGIS extension.

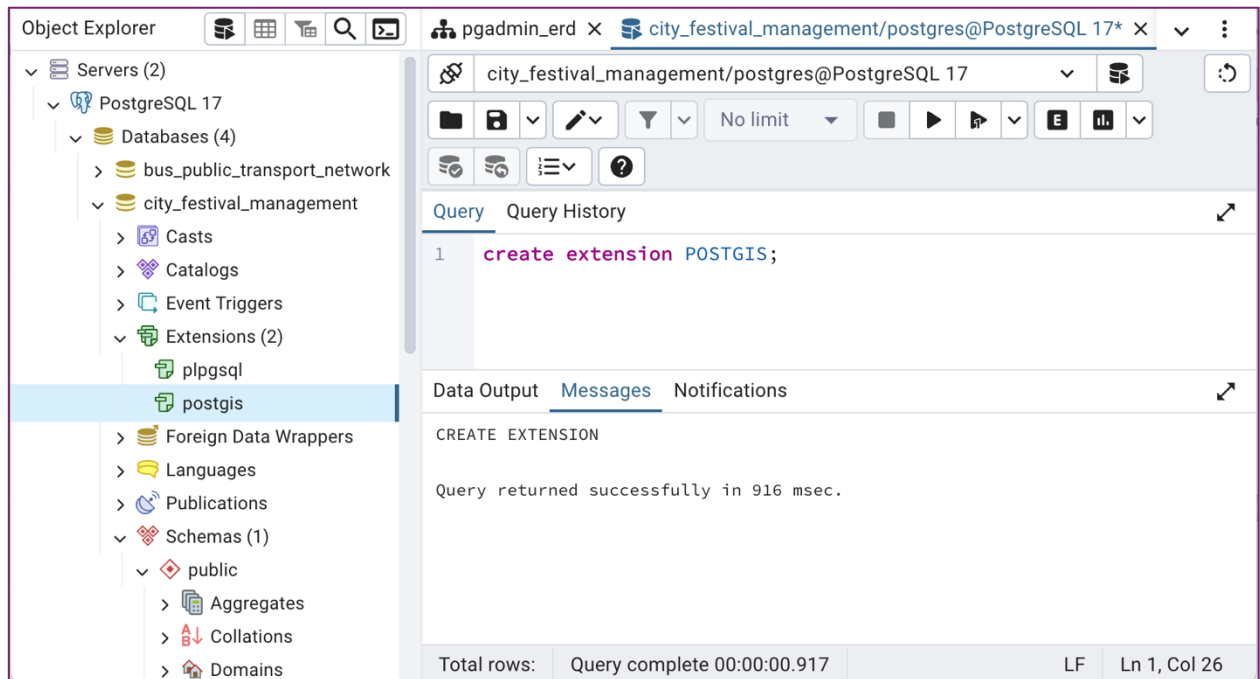


Figure 7: PostGIS Extension Creation

4.2 Connect Database in QGIS

In QGIS, a new PostGIS connection was established for the newly created database, as shown in Figure 8.

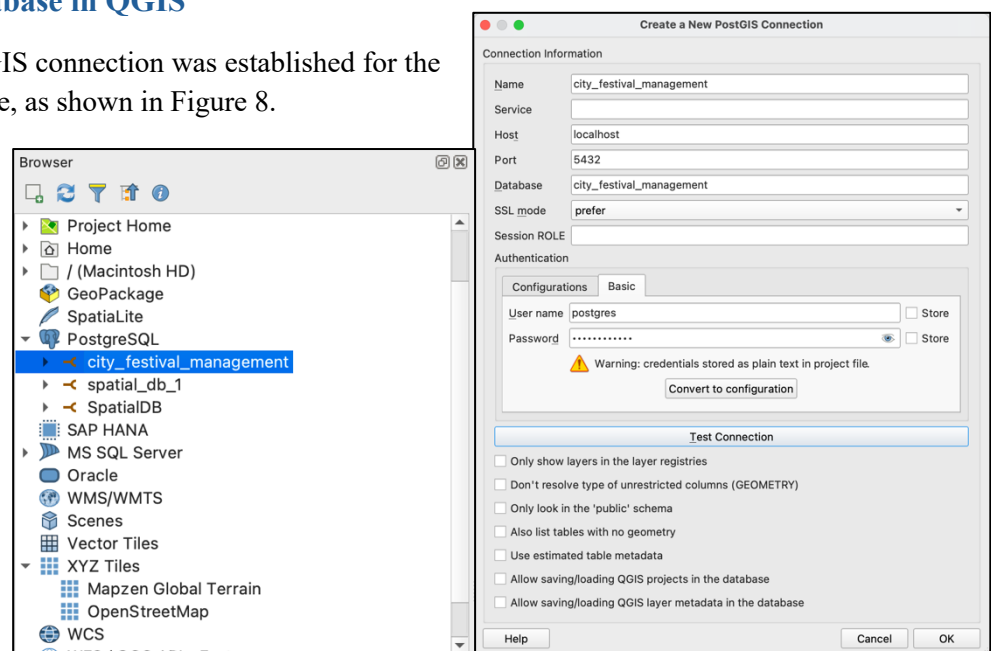
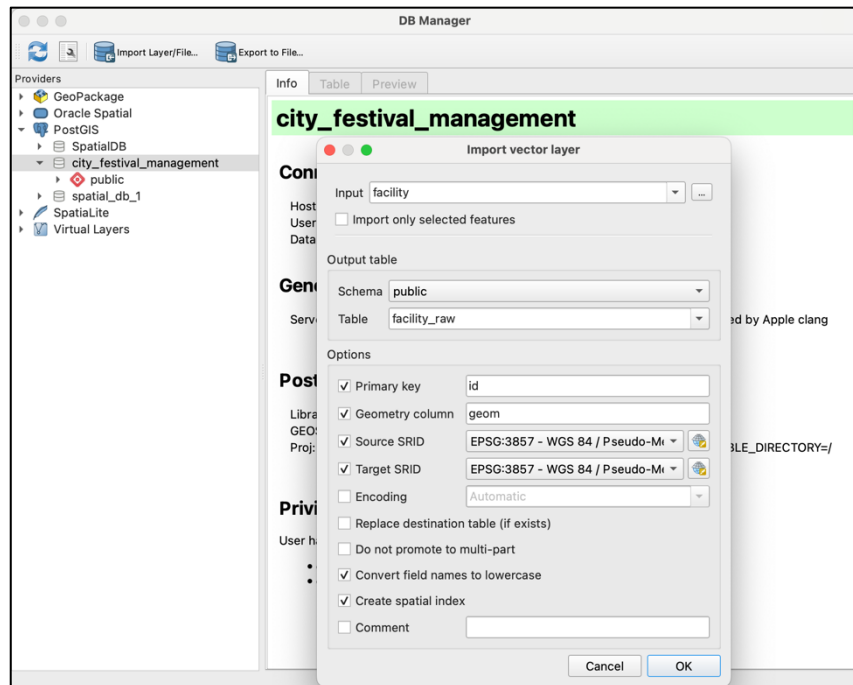


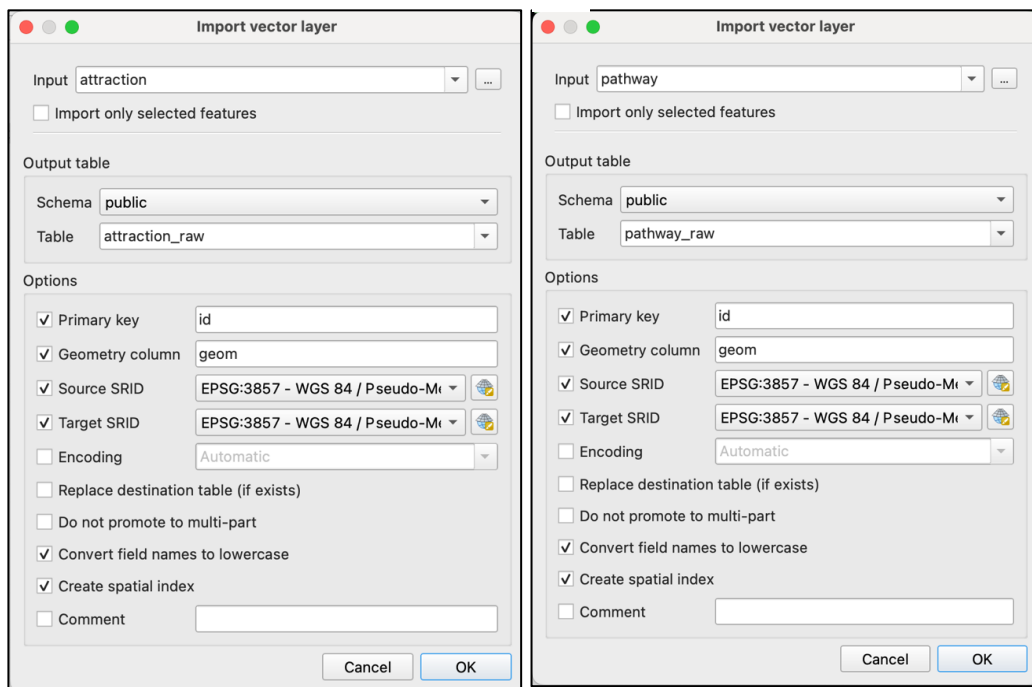
Figure 8: PostGIS Connection in QGIS

4.3 Load Shapefiles into Database

Facility, pathway, and attraction shapefiles were loaded in the database in public schema using DB Manager in QGIS with name 'facility_raw', 'pathway_raw', and 'attraction_raw', respectively as shown in Figure 9.



a



b

c

Figure 9: Import Layers into Database

4.4 Build Entity-Relation Diagram using ERD Tool in Pgadmin4

Using ERD tool in Pgadmin, database was designed. Database tables were defined with columns specified by their data types (e.g., integer, varchar, date, geometry), and and properties such as NOT NULL, and DEFAULT values. In addition to this primary key, unique constraints and foreign key relationships were also established. Figure 10 represents the table definition for **attraction**.

Figure 10 displays the table definition for the **attraction** table in PgAdmin 4, showing the General, Columns, and Constraints tabs.

General Tab:

- Name: attraction
- Schema: public
- Comment: (empty)

Columns Tab:

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id	serial			☒	☒	
code	integer			☒	☐	
name	character varying			☐	☐	
type_id	integer			☒	☐	
start_time	time without time			☐	☐	
end_time	time without time			☐	☐	
geom	geometry			☒	☐	
description	text			☐	☐	

Constraints Tab:

Foreign Key:

Name	Columns	Referenced Table
type_id	(type_id) -> (id)	attraction_type

Unique:

Name	Columns
attraction_code_unique	code

Figure 10: Table Definition in ERD Tool

Figure 11 represents the ER diagram for the city festival management database. The facility table would stores all kinds of facilities available such as toilet, water station, parking etc. Similarly, the attraction table is designed to store all kinds of attractions that includes, including food, gift shop, entertainment, events, etc.. The user table is defined with columns name, address, contact, type_id, and description. Here, contacts is defined with constraints unique contact, which serves as a unique identifier to the contacts and is also used for creating roles. The database is structured to fulfill the third normal form (3NF) to eliminate redundancy and ensure data integrity. For instance, user_type, attraction_type, and facility_type tables are created separately only to store types and are linked with the foreign key in corresponding tables. Likewise, considering one vendor might own multiple stalls/attractions, the "attraction_vendor" table is defined separately. Also, the 'facility_staff' table is designed to handle cases where a single staff member might be assigned to multiple facilities.

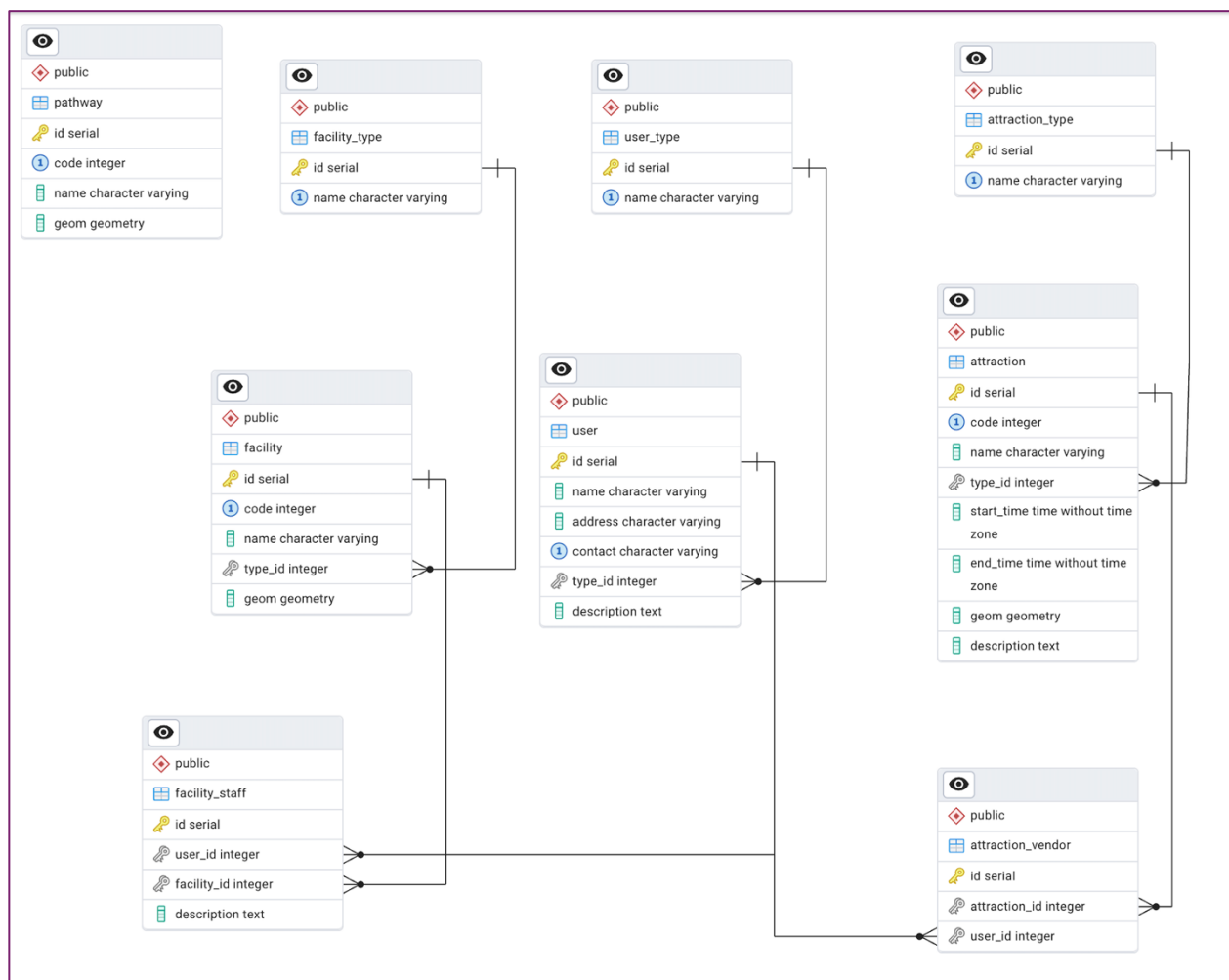


Figure 11: ER Diagram

4.5 Create Tables in Database

After finalizing the ER diagram, SQL was generated as shown below, and SQL was executed to create tables in the database.

```
-- This script was generated by the ERD tool in pgAdmin 4.
-- Please log an issue at https://github.com/pgadmin-
org/pgadmin4/issues/new/choose if you find any bugs, including
reproduction steps.
BEGIN;

CREATE TABLE IF NOT EXISTS public.attraction
(
    id serial NOT NULL,
    code integer NOT NULL,
    name character varying,
    type_id integer NOT NULL,
    start_time time without time zone,
    end_time time without time zone,
    geom geometry NOT NULL,
    description text,
    PRIMARY KEY (id),
    CONSTRAINT attraction_code_unique UNIQUE (code)
);

CREATE TABLE IF NOT EXISTS public.facility
(
    id serial NOT NULL,
    code integer NOT NULL,
    name character varying,
    type_id integer NOT NULL,
    geom geometry NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT facility_code_unique UNIQUE (code)
);

CREATE TABLE IF NOT EXISTS public.pathway
(
    id serial NOT NULL,
    code integer NOT NULL,
    name character varying,
    geom geometry NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT pathway_code_unique UNIQUE (code)
);

CREATE TABLE IF NOT EXISTS public."user"
(
    id serial NOT NULL,
    name character varying NOT NULL,
    address character varying,
    contact character varying NOT NULL,
    type_id integer,
    description text,
    PRIMARY KEY (id),
    CONSTRAINT user_contact_unique UNIQUE (contact)
);

CREATE TABLE IF NOT EXISTS public.facility_type
(
    id serial NOT NULL,
    name character varying NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT facility_name_unique UNIQUE (name)
);
```



```

CREATE TABLE IF NOT EXISTS public.attraction_type
(
    id serial NOT NULL,
    name character varying NOT NULL,
    CONSTRAINT id PRIMARY KEY (id),
    CONSTRAINT attraction_type_name_unique UNIQUE (name)
);

CREATE TABLE IF NOT EXISTS public.attraction_vendor
(
    id serial NOT NULL,
    attraction_id integer NOT NULL,
    user_id integer NOT NULL,
    PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public.facility_staff
(
    id serial NOT NULL,
    user_id integer NOT NULL,
    facility_id integer NOT NULL,
    description text,
    PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public.user_type
(
    id serial NOT NULL,
    name character varying NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT user_type_name_unique UNIQUE (name)
);

ALTER TABLE IF EXISTS public.attraction
    ADD CONSTRAINT type_id FOREIGN KEY (type_id)
    REFERENCES public.attraction_type (id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE RESTRICT;

ALTER TABLE IF EXISTS public.facility
    ADD CONSTRAINT type_id FOREIGN KEY (type_id)
    REFERENCES public.facility_type (id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE RESTRICT;

ALTER TABLE IF EXISTS public."user"
    ADD CONSTRAINT type_id FOREIGN KEY (type_id)
    REFERENCES public.user_type (id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE RESTRICT;

ALTER TABLE IF EXISTS public.attraction_vendor
    ADD CONSTRAINT user_id FOREIGN KEY (user_id)
    REFERENCES public."user" (id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE CASCADE;

ALTER TABLE IF EXISTS public.attraction_vendor
    ADD CONSTRAINT attraction_id FOREIGN KEY (attraction_id)
    REFERENCES public.attraction (id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE CASCADE;

```

```

ALTER TABLE IF EXISTS public.attraction_vendor
  ADD CONSTRAINT attraction_id FOREIGN KEY (attraction_id)
  REFERENCES public.attraction (id) MATCH SIMPLE
  ON UPDATE NO ACTION
  ON DELETE CASCADE;

ALTER TABLE IF EXISTS public.facility_staff
  ADD CONSTRAINT user_id FOREIGN KEY (user_id)
  REFERENCES public."user" (id) MATCH SIMPLE
  ON UPDATE NO ACTION
  ON DELETE CASCADE;

ALTER TABLE IF EXISTS public.facility_staff
  ADD CONSTRAINT facility_id FOREIGN KEY (facility_id)
  REFERENCES public.facility (id) MATCH SIMPLE
  ON UPDATE NO ACTION
  ON DELETE NO ACTION;

END;

```

4.6 Insert Data into Database and Database Normalization

In this step, imported raw layers i.e. `pathway_raw`, `facility_raw` and `attraction_raw`, are inserted into the newly created tables.

Distinct attraction types are retrieved from `attraction_raw` table and added into the `attraction_type` table.

The screenshot shows a database query editor with a query window and a data output window. The query window contains the following SQL code:

```
174 INSERT INTO attraction_type (name)
175
176 SELECT DISTINCT type
177 FROM public.attraction_raw;
178
179 SELECT * FROM attraction_type;
180
```

The data output window shows the results of the query, displaying a table with 5 rows and 2 columns: `id` (integer, primary key) and `name` (character varying). The data is as follows:

	id [PK] integer	name character varying
1	1	Food court
2	2	Entertainment
3	3	Gift shop
4	4	Instaspot
5	5	Event

The status bar at the bottom indicates "Total rows: 5", "Query complete 00:00:00.222", "LF", and "Ln 179, Col 31".

Distinct facility types are retrieved from `facility_raw` table and added into the `facility_type` table.

The screenshot shows a database query editor with a query window and a data output window. The query window contains the following SQL code:

```
182
183 INSERT INTO facility_type (name)
184 SELECT DISTINCT type
185 FROM public.facility_raw;
186
187 SELECT * FROM facility_type;
188
```

The data output window shows the results of the query, displaying a table with 5 rows and 2 columns: `id` (integer, primary key) and `name` (character varying). The data is as follows:

	id [PK] integer	name character varying
1	1	Toilet
2	2	Parking
3	3	Dustbin
4	4	Water station
5	5	Gate

The status bar at the bottom indicates "Total rows: 5", "Query complete 00:00:00.120", "LF", and "Ln 187, Col 29".

Four user types were inserted into the user_type table

Query Query History

```

163 INSERT INTO user_type (name)
164 VALUES
165     ('admin'),
166     ('vendor'),
167     ('staff'),
168     ('visitor'),
169     ('manager');
170
171 SELECT * FROM user_type;
172

```

Data Output Messages Geometry Viewer X Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	id [PK] integer	name character varying
1	1	admin
2	2	vendor
3	3	staff
4	4	visitor
5	5	manager

Total rows: 5 Query complete 00:00:00.124 LF Ln 171, Col 25

Pathway table is populated using the pathway_raw data.

Query Query History

```

189
190 -----
191 INSERT INTO pathway (code, name, geom)
192 SELECT code, name, geom
193 FROM public.pathway_raw;
194
195 SELECT * FROM pathway;|
196
197 -----

```

Data Output Messages Geometry Viewer X Notifications

Showing rows: 1 to 15 Page No: 1 of 1

	integer	code integer	name character varying	geom geometry
1	1	1	Tkacka	0105000020110F000001000000010200000002000000C9282409B5AD3F41E614/
2	2	2	Długa	0105000020110F000001000000010200000004000000A58AA2653AD3F41D9ED
3	3	3	Targ Weglowy	0105000020110F00000100000001020000000D00000068AE3C96D7AB3F410505
4	4	4	Waly Jagiellonskie	0105000020110F000001000000010200000002000000BF22EF92E8AB3F412A44
5	5	5	Waly Jagiellonskie	0105000020110F000001000000010200000002000000EE4EE55E1CAC3F4111DF
6	6	6	Targ Sienny	0105000020110F000001000000010200000004000000B31E23F9C5AB3F413AB4
7	7	7	Targ Weglowy	0105000020110F00000100000001020000000600000034F331B992AC3F41514B

Total rows: 15 Query complete 00:00:00.187 LF Ln 195, Col 23

Facility table is populated using the facility_raw data and type_id (FK) is extracted from facility_type table.

Query Query History

```

198
199 v INSERT INTO facility (code, name, geom, type_id)
200 SELECT fr.code, fr.name, fr.geom, ft.id AS type_id
201 FROM facility_raw as fr
202 JOIN
203     facility_type as ft
204 ON
205     fr.type = ft.name;
206
207 SELECT * FROM facility;
208

```

Data Output Messages Geometry Viewer X Notifications

Showing rows: 1 to 15 Page No: 1 of 1

	id [PK] integer	code integer	name character varying	type_id integer	geom geometry
1	1	3	WC2	1	0101000020110F0000470E629DDBAC3F4116753FE0509B5B4
2	2	2	WC1	1	0101000020110F0000C819A376B5AC3F41363B1D1E009B5B4
3	3	5	P3	2	0101000020110F00004C96023BFCAB3F4189A267F1389B5B4
4	4	4	P2	2	0101000020110F00004F4CB78E7CAC3F41EB6F53D0F09A5B4
5	5	1	P1	2	0101000020110F00002BA7F83FE5AC3F41006FFD845B9B5B4
6	6	7	DB4	3	0101000020110F0000BDE71AFBD5AC3F41A9C7F70A4D9B5B.

Total rows: 15 Query complete 00:00:00.053 LF Ln 206, Col 1

Similarly, attraction table is populated using the attraction_raw data and type_id (FK) is extracted from attraction_type.

Query Query History

```

209 -----
210 v INSERT INTO attraction (code, name, start_time, end_time, geom, type_id)
211 SELECT ar.code, ar.name, ar.start_time::time, ar.end_time::time, ar.geom, at.id AS type_id
212 FROM attraction_raw as ar
213 JOIN
214     attraction_type as at
215 ON
216     ar.type = at.name;
217
218 SELECT * FROM attraction;
219

```

Data Output Messages Geometry Viewer X Notifications

Showing rows: 1 to 159 Page No: 1 of 1

	id [PK] integer	code integer	name character varying	type_id integer	start_time time without time zone	end_time time without time zone	geom geometry
14	14	14	Food court 2	1	17:00:00	22:00:00	0106C
15	15	158	Food court 84	1	17:00:00	22:00:00	0106C
16	16	16	Food court 3	1	17:00:00	22:00:00	0106C
17	17	60	Food court 33	1	17:00:00	22:00:00	0106C
18	18	17	Scene - The Glorious Throne of Santa Claus	5	17:00:00	20:00:00	0106C
19	19	18	Snapshot with GPEC Group	5	17:00:00	20:00:00	0106C
20	20	61	Advent Gate 1	5	17:00:00	20:00:00	0106C
21	21	20	Group with a Chapel	5	17:00:00	20:00:00	0106C

Total rows: 159 Query complete 00:00:00.064 LF Ln 216, Col 23

For this case study, contact info is considered a unique identifier of the vendors. Thus, the user table is populated based on unique contacts from the attraction table.

Query

Query History

221

222

223

224

225

226

227

228

INSERT INTO public.user (name,address, contact,type_id)

SELECT DISTINCT ON (ven_con) ven_name,ven_adr,ven_con, 2

FROM public.attraction_raw;

SELECT * FROM public.user;

Data Output

Messages

Geometry Viewer

Notifications

SQL

Showing rows: 1 to 157

Page No: 1

of 1

	id [PK] integer	name character varying	address character varying	contact character varying	type_id integer	description text
1	1	Michele Drake	6374 Robinson Corners, Port Ri...	+43 6104182659	2	[null]
2	2	Jared Thompson	6794 Lisa Shores Suite 018, Por...	+43 6120500448	2	[null]
3	3	Henry Singh	30828 Kelly Fort Suite 160, Port ...	+43 6126686701	2	[null]
4	4	Shari Lopez	481 Hogan Locks Suite 125, Bar...	+43 6131832271	2	[null]
5	5	Maria Wood	481 Barbara Tunnel Apt. 894, Ca...	+43 6133784345	2	[null]
6	6	Garrett Collins	PSC 4259, Box 5976, APO AA 0...	+43 6137499744	2	[null]
7	7	Stephen Mason	123 Davis Mill Apt. 421, New Mi...	+43 6141812512	2	[null]
8	8	Vincent Huynh	07002 Bradley Mountain Suite 5...	+43 6145330324	2	[null]
9	9	Christina Hamilton	3736 Maurice Glen, Davidfurt, N...	+43 6147973512	2	[null]

Total rows: 157

Query complete 00:00:00.135

LF

Ln 228, Col 65

Based on the contacts in attraction_raw and user table, the attraction_vendor table is populated.

Query

Query History

231

▼

INSERT INTO attraction_vendor (attraction_id, user_id)

232

SELECT ar.id as attraction_id, u.id as user_id

233

FROM attraction_raw as ar

234

JOIN

235

public.user as u

236

ON

237

u.contact = ar.ven_con

238

ORDER BY u.id;

239

240

SELECT * FROM attraction_vendor;

...

Data Output

Messages

Geometry Viewer

X

Notifications

≡

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

Showing rows: 1 to 159

✎

 Page No: 1 of 1

⏪

⏴

⏵

⏩

	id [PK] integer ✎	attraction_id integer ✎	user_id integer ✎
1	1	136	1
2	2	63	2
3	3	144	3
4	4	151	4
5	5	152	5
6	6	46	6

Total rows: 159

Query complete 00:00:00.166

LF

Ln 235, Col 21

The user table consist of only vendors, some other users are added into the table. Here, type_id (FK) determines the type of the user, which is based on user_type table.

Query Query History

```

243
244 INSERT INTO public.user (name,address,contact,type_id)
245 VALUES
246 ('Ram Ghimire','6374 Robinson Corners, Port Rick','+43 6104182611',3),
247 ('Pragya Pant','6375 Robinson Corners, Port Rick','+43 6104182622',3),
248 ('Manisha Shrestha', '6376 Robinson Corners, Port Rick','+43 6104182633',3),
249 ('Dinesh Bhatt','6377 Robinson Corners, Port Rick','+43 6104182644',3)
250 ('Visitor 1','6374 Robinson Corners, Port Rick','+43 6104182655',4),
251 ('Visitor 2','6374 Robinson Corners, Port Rick','+43 6104182656',4),
252 ('Manager 1','6374 Robinson Corners, Port Rick','+43 6104182669',5),
253 ('Manager 2','6374 Robinson Corners, Port Rick','+43 6104182668',5);
254
255 SELECT * FROM public.user where type_id=3;
256
---
```

Data Output Messages Geometry Viewer X Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	id [PK] integer	name character varying	address character varying	contact character varying	type_id integer	description text
1	150	Ram Ghimire	6374 Robinson Corners, Port Rick	+43 6104182611	3	[null]
2	151	Pragya Pant	6375 Robinson Corners, Port Rick	+43 6104182622	3	[null]
3	152	Manisha Shrestha	6376 Robinson Corners, Port Rick	+43 6104182633	3	[null]
4	153	Dinesh Bhatt	6377 Robinson Corners, Port Rick	+43 6104182644	3	[null]

Total rows: 4 Query complete 00:00:00.209 LF Ln 244, Col 1

Based on user table and facility table, following values are inserted into facility_staff table that define the role for particular staff.

Query Query History

```

259 INSERT INTO facility_staff (facility_id,user_id)
260 VALUES
261 (1,150),
262 (2,150),
263 (5,151),
264 (4,152),
265 (6,153);
266
267 select * from public.user where type_id=3;
268 select * from public.facility;
269
270 select * from facility_staff;
271
---
```

Data Output Messages Geometry Viewer X Notifications

Showing rows: 1 to 6 Page No: 1 of 1

	id [PK] integer	user_id integer	facility_id integer	description text
1	1	150	1	[null]
2	2	150	2	[null]
3	3	151	5	[null]
4	4	152	4	[null]
5	5	153	6	[null]
6	6	153	3	[null]

Total rows: 6 Query complete 00:00:00.086 LF Ln 267, Col 43

4.7 Create View

The following views were created. Attraction_view and facility_view are again denormalized by defining the column type based on the type_id (FK) field.

```
CREATE VIEW attraction_view AS
  SELECT
    a.id,a.name,a.start_date,a.end_date,a.start_time,a.end_time,a.geom,a.d
    escription,at.name as type
  FROM
    attraction as a
  JOIN
    attraction_type as at
  ON
    a.type_id=at.id;

CREATE VIEW facility_view AS
  SELECT
    f.id,f.name,f.geom,ft.name as type
  FROM
    facility as f
  JOIN
    facility_type as ft
  ON
    f.type_id=ft.id;

CREATE VIEW pathway_view AS
  SELECT
    *
  FROM
    pathway;
```

4.8 Define Roles and Permissions

The **visitor role** was created for one of the visitor users using a contact number, which is a unique column. Only view access is provided for the `attraction_view`, `facility_view`, `pathway_view`, `attraction_type`, and `facility_type`.

```
CREATE ROLE visitor_6104182656 LOGIN PASSWORD '6104182656';

DO $$
DECLARE
    role_name TEXT := 'visitor_6104182656';
BEGIN
    EXECUTE format('REVOKE ALL ON attraction_view FROM %I', role_name);
    EXECUTE format('GRANT SELECT ON attraction_view TO %I', role_name);

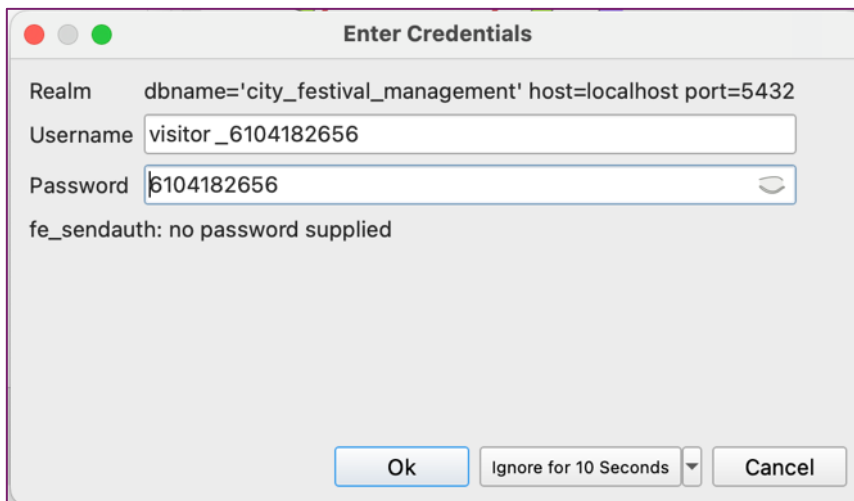
    EXECUTE format('REVOKE ALL ON facility_view FROM %I', role_name);
    EXECUTE format('GRANT SELECT ON facility_view TO %I', role_name);

    EXECUTE format('REVOKE ALL ON pathway_view FROM %I', role_name);
    EXECUTE format('GRANT SELECT ON pathway_view TO %I', role_name);

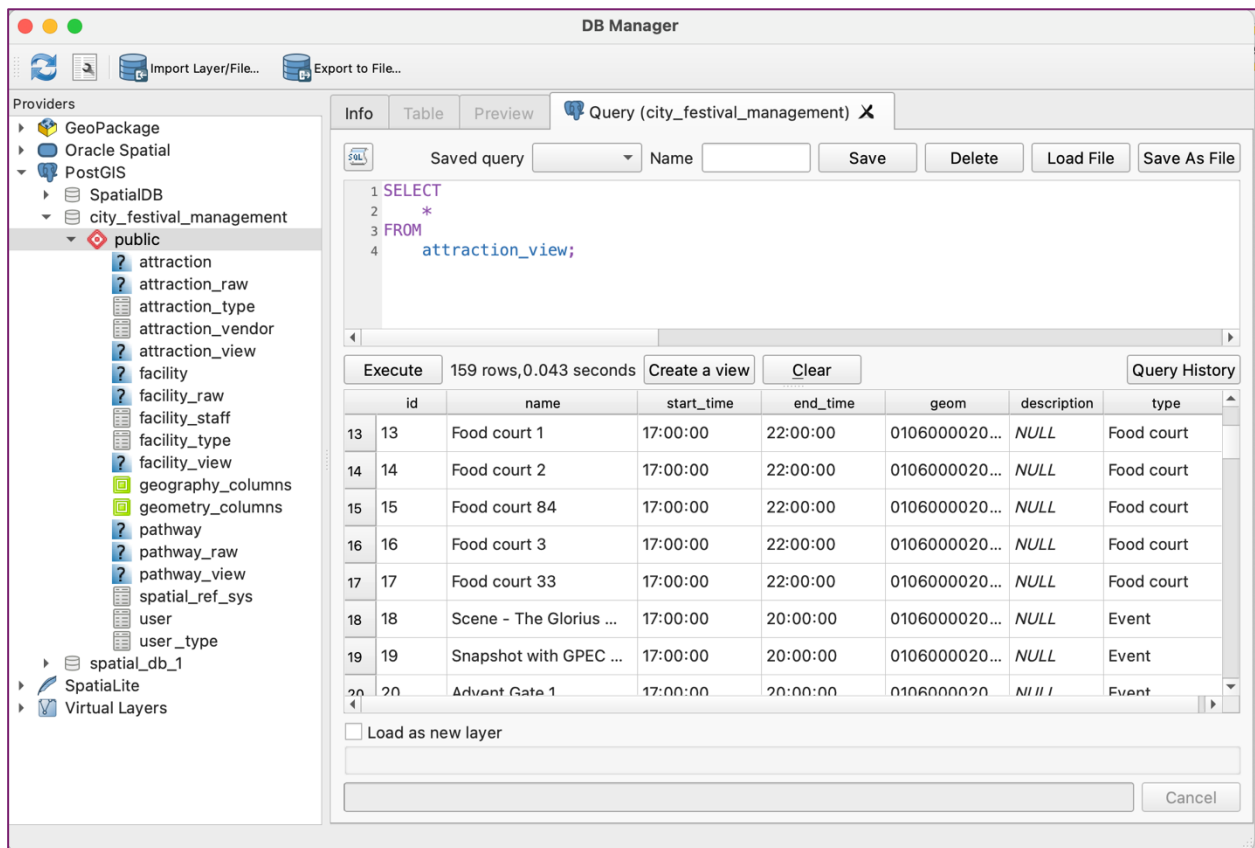
    EXECUTE format('REVOKE ALL ON attraction_type FROM %I', role_name);
    EXECUTE format('GRANT SELECT ON attraction_type TO %I', role_name);

    EXECUTE format('REVOKE ALL ON facility_type FROM %I', role_name);
    EXECUTE format('GRANT SELECT ON facility_type TO %I', role_name);
END $$;
```

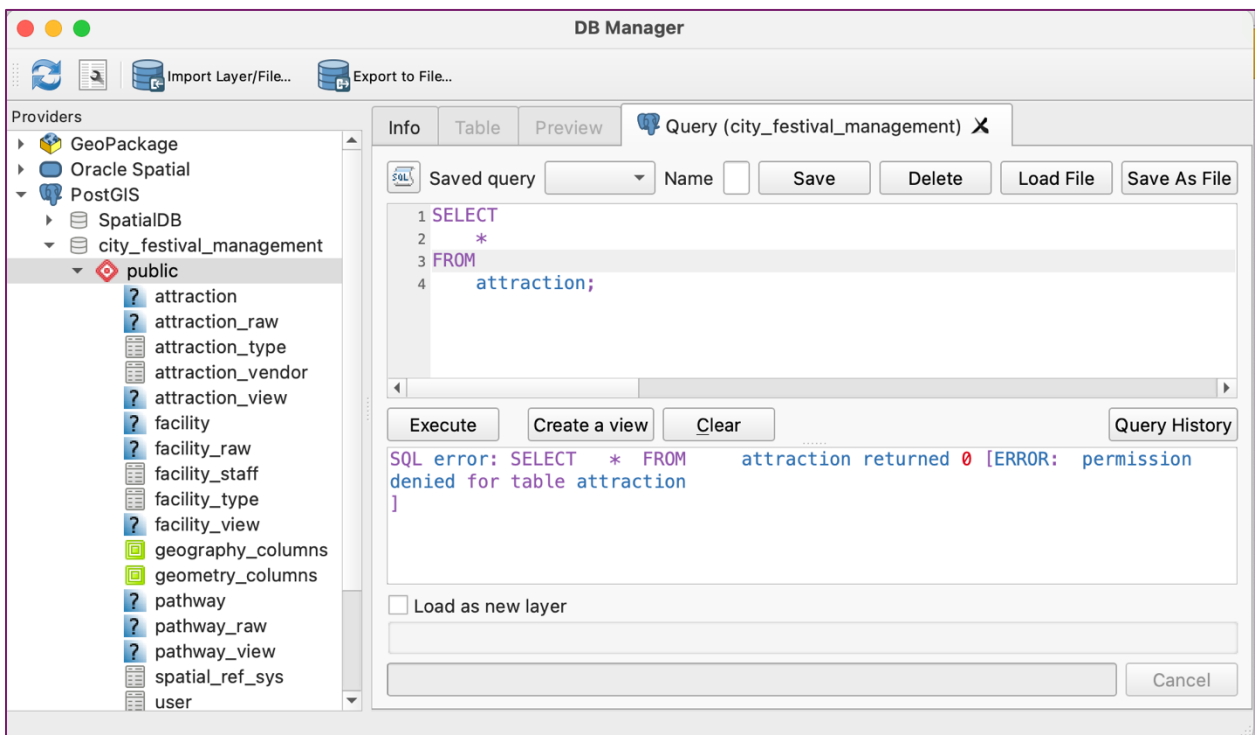
Then, the database is connected with the visitor credentials as shown below.



As a visitor, the attraction view is accessible.



But, the attraction table is not accessible as visitors have no permission to access this table.



Likewise, the **manager role** was created for the manager user added in the user table with permission, as shown below.

```
CREATE ROLE manager_6104182668 LOGIN PASSWORD '6104182668';
DO $$
DECLARE
    role_name TEXT := 'manager_6104182668';
BEGIN
    EXECUTE format('REVOKE ALL ON attraction FROM %I', role_name);
    EXECUTE format('GRANT USAGE, SELECT ON SEQUENCE attraction_id_seq TO %I', role_name);
    EXECUTE format('GRANT SELECT, INSERT, UPDATE, DELETE ON attraction TO %I', role_name);

    EXECUTE format('REVOKE ALL ON facility FROM %I', role_name);
    EXECUTE format('GRANT USAGE, SELECT ON SEQUENCE facility_id_seq TO %I', role_name);
    EXECUTE format('GRANT SELECT, INSERT, UPDATE, DELETE ON facility TO %I', role_name);

    EXECUTE format('REVOKE ALL ON pathway FROM %I', role_name);
    EXECUTE format('GRANT USAGE, SELECT ON SEQUENCE pathway_id_seq TO %I', role_name);
    EXECUTE format('GRANT SELECT, INSERT, UPDATE, DELETE ON pathway TO %I', role_name);

    EXECUTE format('REVOKE ALL ON attraction_type FROM %I', role_name);
    EXECUTE format('GRANT SELECT ON attraction_type TO %I', role_name);

    EXECUTE format('REVOKE ALL ON facility_type FROM %I', role_name);
    EXECUTE format('GRANT SELECT ON facility_type TO %I', role_name);

    EXECUTE format('REVOKE ALL ON public.user FROM %I', role_name);
    EXECUTE format('GRANT SELECT ON public.user TO %I', role_name);

    EXECUTE format('REVOKE ALL ON attraction_vendor FROM %I', role_name);
    EXECUTE format('GRANT USAGE, SELECT ON SEQUENCE attraction_vendor_id_seq TO %I', role_name);
    EXECUTE format('GRANT SELECT, INSERT, UPDATE, DELETE ON attraction_vendor TO %I', role_name);

    EXECUTE format('REVOKE ALL ON facility_staff FROM %I', role_name);
    EXECUTE format('GRANT USAGE, SELECT ON SEQUENCE facility_staff_id_seq TO %I', role_name);
    EXECUTE format('GRANT SELECT, INSERT, UPDATE, DELETE ON facility_staff TO %I', role_name);

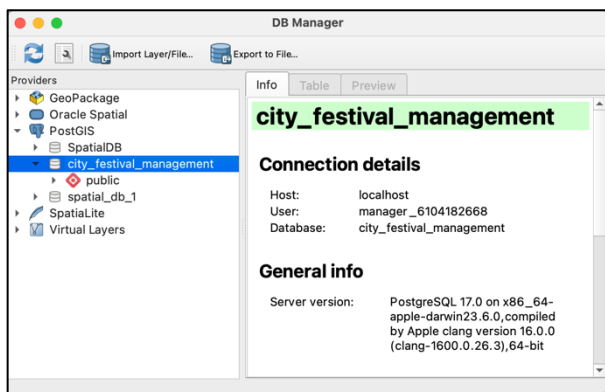
    EXECUTE format('REVOKE ALL ON attraction_view FROM %I', role_name);
    EXECUTE format('GRANT SELECT ON attraction_view TO %I', role_name);

    EXECUTE format('REVOKE ALL ON facility_view FROM %I', role_name);
    EXECUTE format('GRANT SELECT ON facility_view TO %I', role_name);

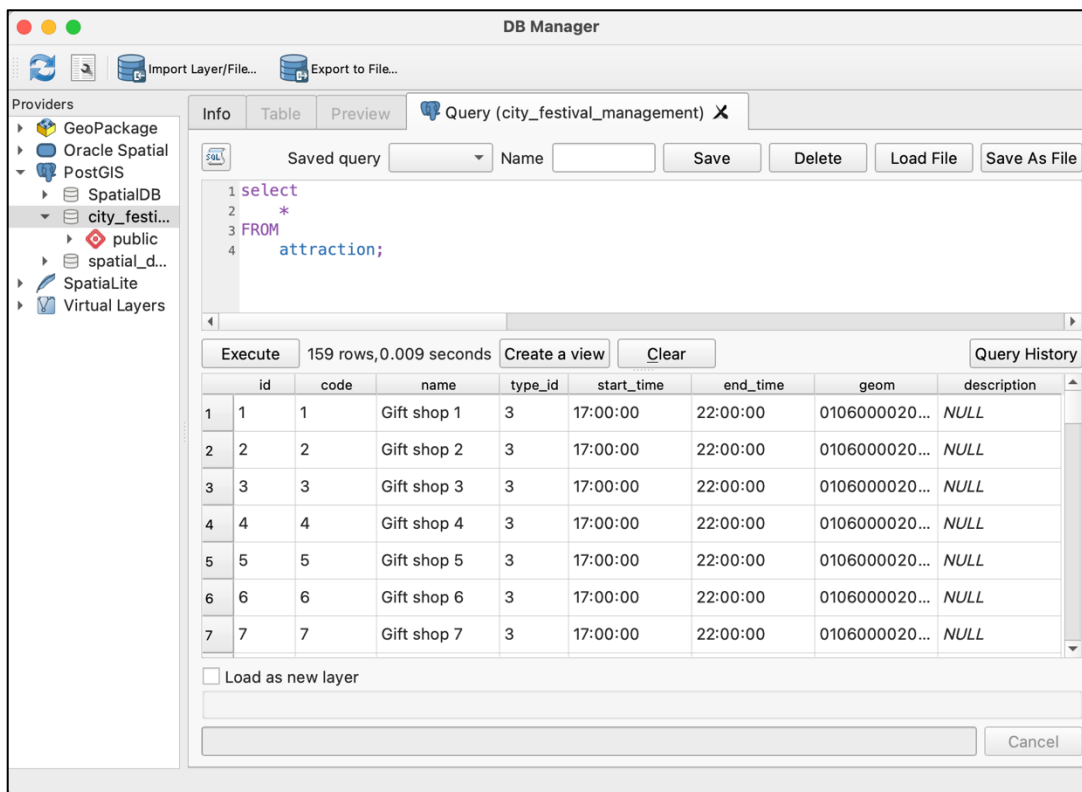
    EXECUTE format('REVOKE ALL ON pathway_view FROM %I', role_name);
    EXECUTE format('GRANT SELECT ON pathway_view TO %I', role_name);

END $$;
```

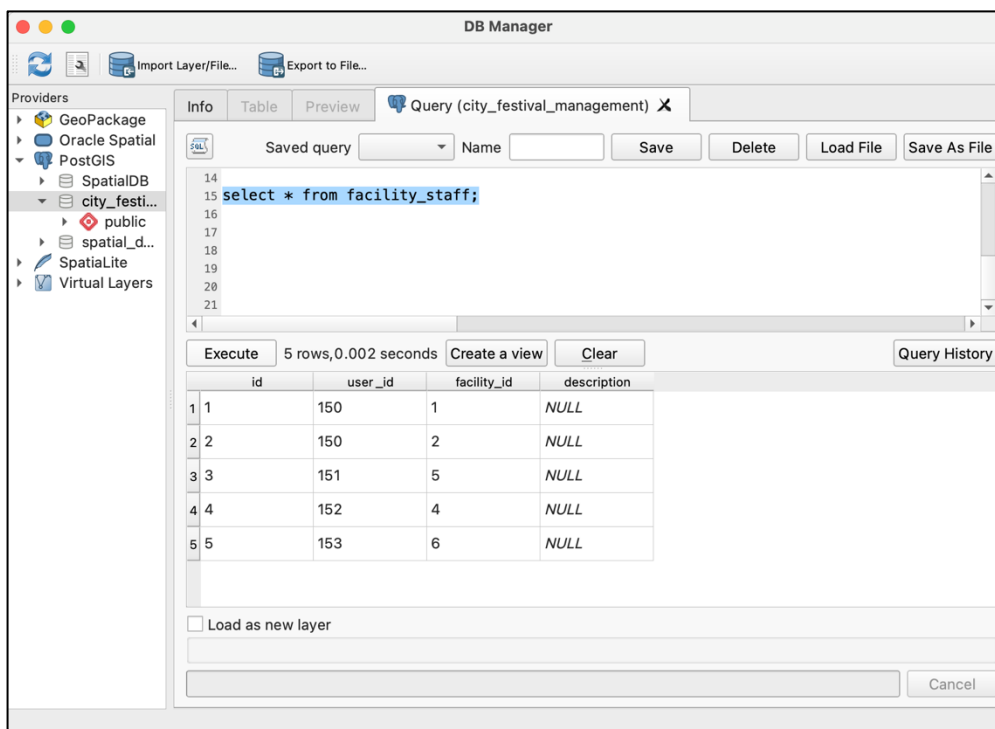
Then, database is connected with the visitor credential as shown below.

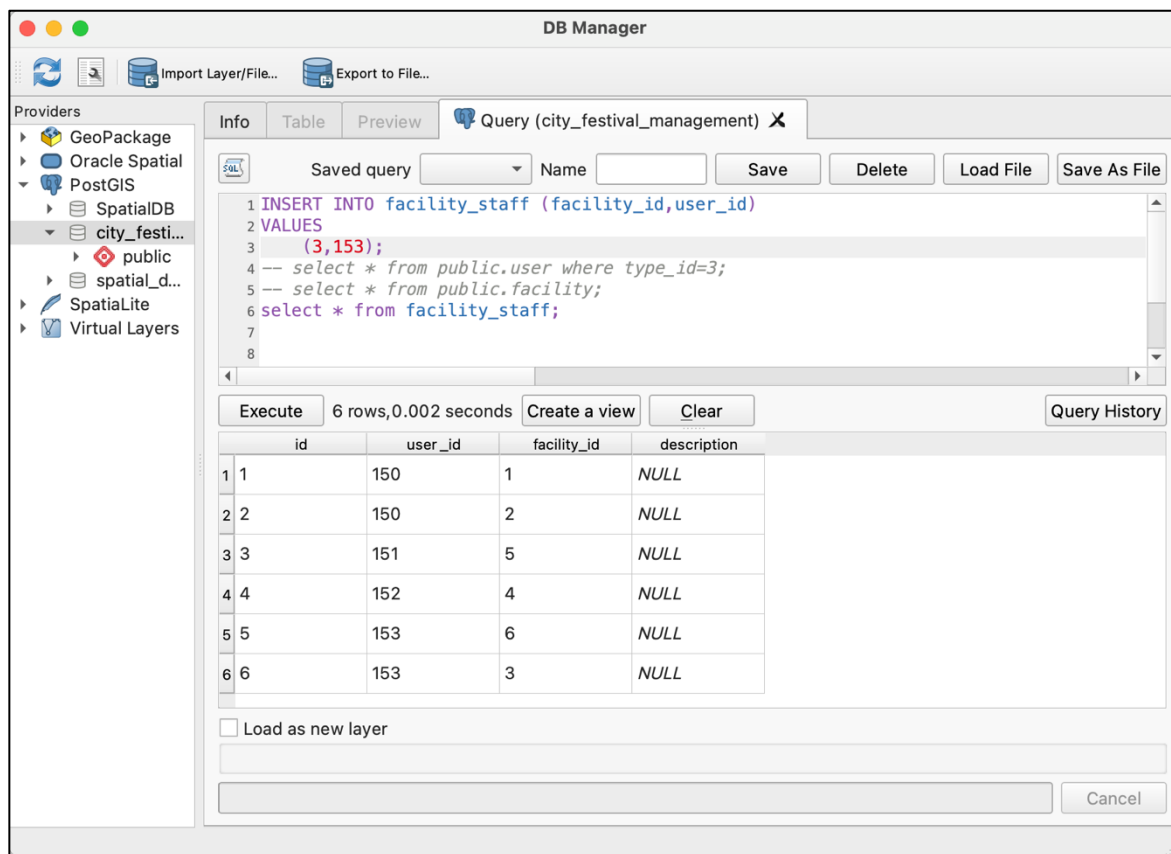


As a manager, the user can access the tables and views as well.



Manager role users also can add, update, and delete the records. For instance, the manager can add the information regarding the staff and facility responsibility, as shown below.





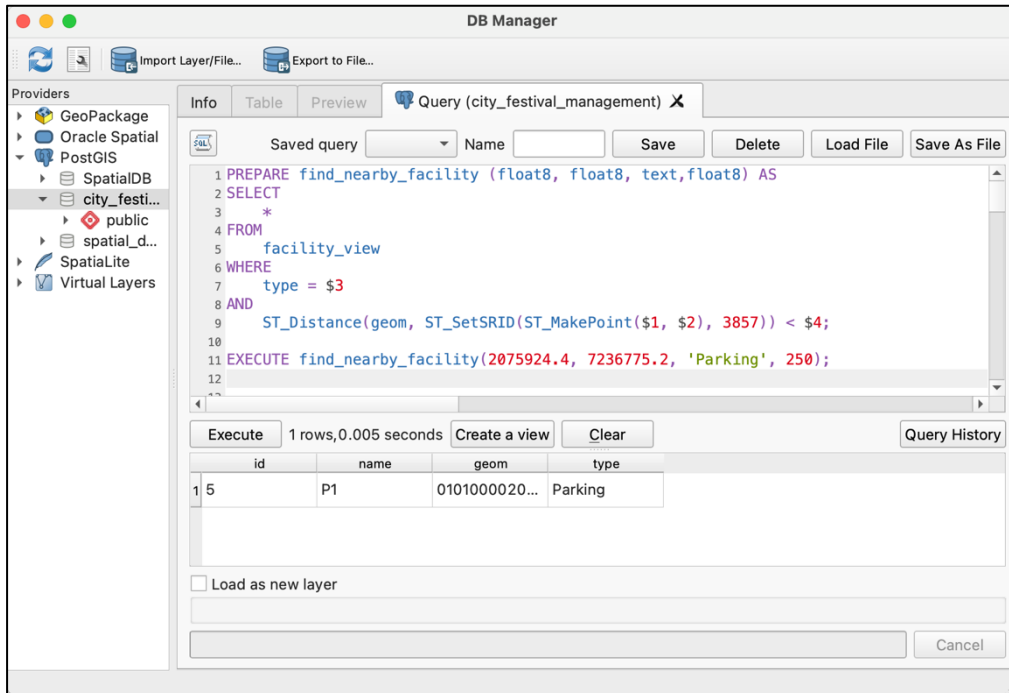
In this way, users with manager roles can manipulate the data with the database depending on the access given to the particular tables.

5 QUERIES

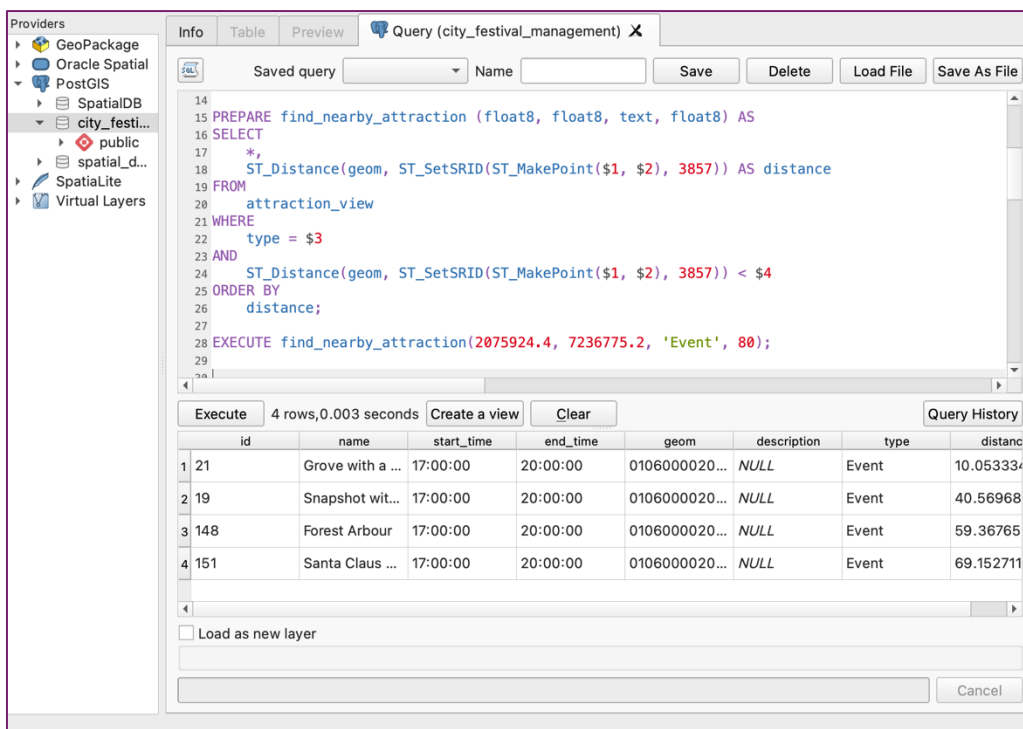
5.1 Queries based on User's Location

- a. To find the list of nearby facilities based on user's location, facility_type, and distance parameter.

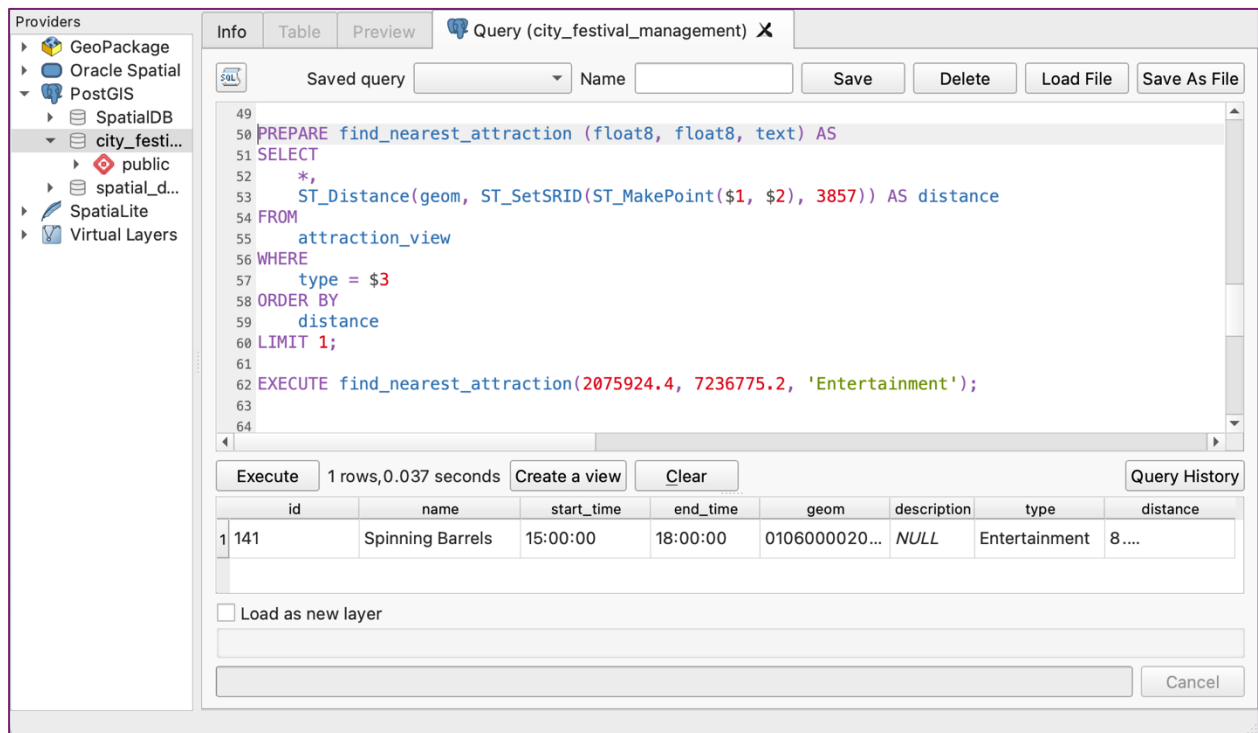
PREPARE statement was defined that takes location (lat & lon), facility_type, and within_distance value as parameters and return the list of nearby facilities.



- b. To find the list of nearby attractions based on user's location, attraction_type, and distance parameter.



c. To find the nearest attraction based on the user's location and attraction_type parameter



The screenshot shows a GIS application interface with a sidebar on the left listing providers: GeoPackage, Oracle Spatial, PostGIS, SpatialDB, city_festi..., public, spatial_d..., SpatiaLite, and Virtual Layers. The main window has tabs for Info, Table, Preview, and Query (city_festival_management). The Query tab is active, showing a SQL query in a text editor. Below the editor, there are buttons for Execute, Create a view, Clear, and Query History. The results of the query are displayed in a table below the buttons.

```

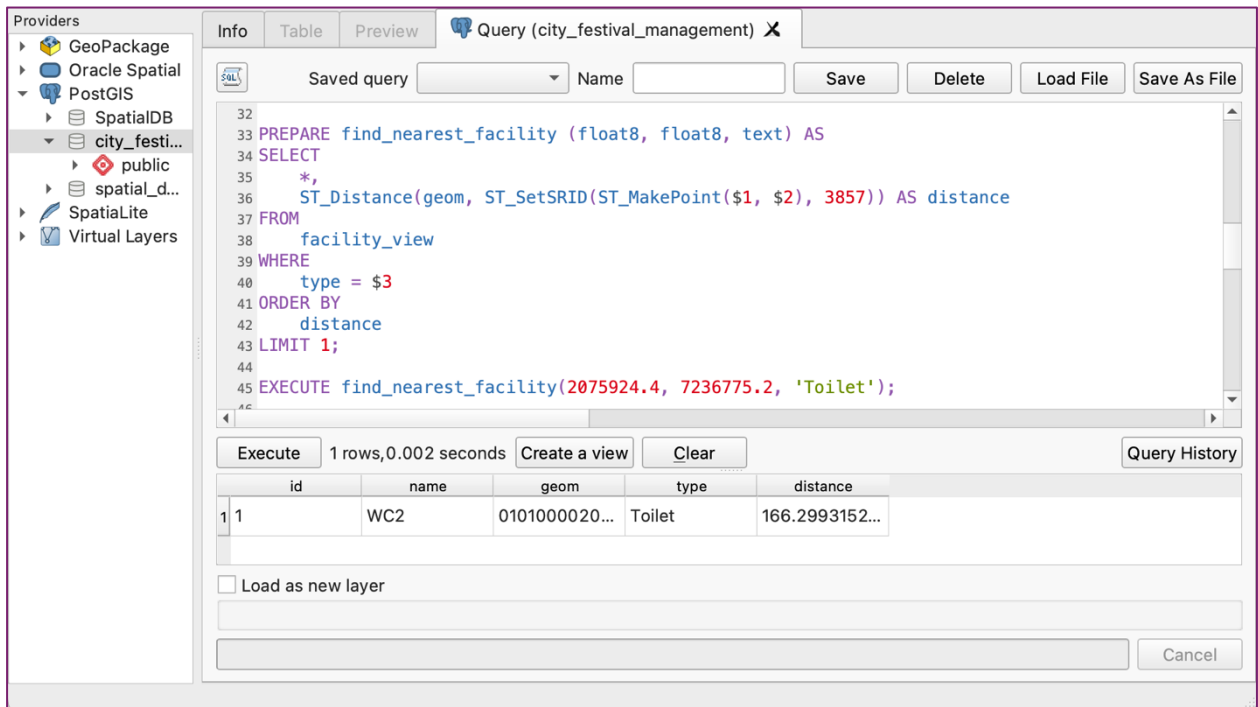
49
50 PREPARE find_nearest_attraction (float8, float8, text) AS
51 SELECT
52 *,
53 ST_Distance(geom, ST_SetSRID(ST_MakePoint($1, $2), 3857)) AS distance
54 FROM
55 attraction_view
56 WHERE
57 type = $3
58 ORDER BY
59 distance
60 LIMIT 1;
61
62 EXECUTE find_nearest_attraction(2075924.4, 7236775.2, 'Entertainment');
63
64

```

id	name	start_time	end_time	geom	description	type	distance
1	141	Spinning Barrels	15:00:00	18:00:00	0106000020...	NULL	Entertainment 8....

Below the table, there is a checkbox labeled "Load as new layer" and a "Cancel" button.

d. To find the nearest facility based on user's location and facility_type parameter



The screenshot shows the same GIS application interface as above. The Query tab is active, showing a SQL query in a text editor. Below the editor, there are buttons for Execute, Create a view, Clear, and Query History. The results of the query are displayed in a table below the buttons.

```

32
33 PREPARE find_nearest_facility (float8, float8, text) AS
34 SELECT
35 *,
36 ST_Distance(geom, ST_SetSRID(ST_MakePoint($1, $2), 3857)) AS distance
37 FROM
38 facility_view
39 WHERE
40 type = $3
41 ORDER BY
42 distance
43 LIMIT 1;
44
45 EXECUTE find_nearest_facility(2075924.4, 7236775.2, 'Toilet');
46

```

id	name	geom	type	distance	
1	1	WC2	0101000020...	Toilet	166.2993152...

Below the table, there is a checkbox labeled "Load as new layer" and a "Cancel" button.

5.2 Queries not based on User's Location

a. To get the specific attraction or facility

The screenshot shows the QGIS Query Manager window for a query named 'city_festival_management'. The query is a prepared statement that selects from the 'attraction_view' table where the name is 'Forest Arbour'. The query is executed, and the results are displayed in a table with 7 columns: id, name, start_time, end_time, geom, description, and type. The results show one row with id 148, name 'Forest Arbour', start_time '17:00:00', end_time '20:00:00', geom '0106000020...', description 'NULL', and type 'Event'.

```
64
65
66 PREPARE find_specified_attraction (text) AS
67 SELECT
68 *
69 FROM
70 attraction_view
71 WHERE
72 name = $1;
73
74 EXECUTE find_specified_attraction('Forest Arbour');
```

	id	name	start_time	end_time	geom	description	type
1	148	Forest Arbour	17:00:00	20:00:00	0106000020...	NULL	Event

b. To get list of facility types

The screenshot shows the QGIS Query Manager window for a query named 'city_festival_management'. The query is a simple SELECT statement that selects all rows from the 'facility_type' table. The query is executed, and the results are displayed in a table with 2 columns: id and name. The results show 5 rows with id values 1 through 5 and names 'Toilet', 'Parking', 'Dustbin', 'Water station', and 'Gate'.

```
1 SELECT
2 *
3 FROM
4 facility_type;
5
6
7
```

	id	name
1	1	Toilet
2	2	Parking
3	3	Dustbin
4	4	Water station
5	5	Gate

c. To get list of attraction types

The screenshot shows the QGIS Query Manager window for a query named 'city_festival_management'. The query is a simple SELECT statement: `SELECT * FROM attraction_type;`. The results are displayed in a table with two columns: 'id' and 'name'. The results show five rows of data.

	id	name
1	1	Food court
2	2	Entertainment
3	3	Gift shop
4	4	Instaspot
5	5	Event

d. To get all the lists of attractions or facilities based on type

The screenshot shows the QGIS Query Manager window for a query named 'city_festival_management'. The query is a SELECT statement with a WHERE clause: `SELECT id, name, start_time, end_time FROM attraction_view WHERE type='Gift shop';`. The results are displayed in a table with four columns: 'id', 'name', 'start_time', and 'end_time'. The results show seven rows of data, all for 'Gift shop' type attractions.

	id	name	start_time	end_time
1	1	Gift shop 1	17:00:00	22:00:00
2	2	Gift shop 2	17:00:00	22:00:00
3	3	Gift shop 3	17:00:00	22:00:00
4	4	Gift shop 4	17:00:00	22:00:00
5	5	Gift shop 5	17:00:00	22:00:00
6	6	Gift shop 6	17:00:00	22:00:00
7	7	Gift shop 7	17:00:00	22:00:00

6 PGROUTING IMPLEMENTATION

6.1 Installation and create extension

Following the steps mentioned in this [source](#), pgrouting extension was installed as follows:

```
>> git clone https://github.com/pgRouting/pgrouting.git
>> cd pgrouting
>> mkdir build
>> cd build
>> cmake ..
>> make
>> sudo make install
>> /Library/PostgreSQL/17/bin/pg_ctl -D /Library/PostgreSQL/17/data restart
```

```
CREATE EXTENSION pgrouting;
```

6.2 Building Routing Topology

Multilinestring geometry type was not supported in pgrouting, thus pathway_edges table of linetrans geometry data type was created from pathway table as follows:

```
SELECT DISTINCT ST_GeometryType(geom)
FROM pathway;

CREATE TABLE pathway_edges (
    id SERIAL PRIMARY KEY,
    name TEXT,
    geom GEOMETRY(LINESTRING, 3857) NOT NULL
);

INSERT INTO pathway_edges (name, geom)
SELECT name, (ST_Dump(geom)).geom AS geom
FROM pathway;
```

Since all the facility points are not linked with the pathway, following SQL was executed to create a new line that connects the facility points to the nearest pathway.

```
INSERT INTO pathway_edges (geom)
SELECT
    ST_MakeLine(facility.geom,
        (SELECT ST_ClosestPoint(roads.geom, facility.geom)
         FROM pathway_edges AS roads
         ORDER BY ST_Distance(roads.geom, facility.geom)
         LIMIT 1)
        ) AS geom
FROM facility;
```

Similarly, to connect attractions to the nearest pathway, centroid of the polygon is created and connects between centroid and nearest pathway as follow:

```
ALTER TABLE attraction
ADD COLUMN centroid geometry(Point, 3857);

UPDATE attraction
SET centroid = ST_Centroid(geom);

INSERT INTO pathway_edges (geom)
SELECT
    ST_MakeLine(attraction.centroid,
        (SELECT ST_ClosestPoint(roads.geom, attraction.centroid)
         FROM pathway_edges AS roads
         ORDER BY ST_Distance(roads.geom, attraction.centroid)
         LIMIT 1)
        ) AS geom
FROM attraction;
```

In total, 189 edges is formed that contain original pathways, connections between facilities and pathways, and attractions and pathways.

The screenshot displays a database query interface. The top toolbar contains various icons for file operations, query execution, and data management. Below the toolbar, the 'Query' tab is active, showing a SQL query in a text editor. The query is as follows:

```
select * from pathway_edges;
```

Below the query editor, the 'Data Output' tab is active, displaying the results of the query. The results are shown in a table with the following columns: 'id' (integer, primary key), 'name' (text), and 'geom' (geometry). The table contains 189 rows, with the first 8 rows visible in the screenshot. The status bar at the bottom indicates 'Total rows: 189', 'Query complete 00:00:00.169', and the current position 'Ln 615, Col 1'.

	id [PK] integer	name text	geom geometry
1	1	Tkacka	0102000020110F0000002000000C9282409B5AD3F41E614A27C489B5B4
2	2	Długa	0102000020110F00000040000000A58AA2653AD3F41D9ED03E7029B5B4
3	3	Targ Weglowy	0102000020110F000000D00000068AE3C96D7AB3F4105057CB5219B5B4
4	4	Waly Jagiellonskie	0102000020110F0000002000000BF22EF92E8AB3F412A4419E61F9B5B41
5	5	Waly Jagiellonskie	0102000020110F0000002000000EE4EE55E1CAC3F4111DF4B541A9B5B4
6	6	Targ Sienny	0102000020110F0000004000000B31E23F9C5AB3F413AB4E182199B5B4
7	7	Targ Weglowy	0102000020110F000000600000034F331B992AC3F41514B57691F9B5B41
8	8	Trag Weglowy	0102000020110F0000002000000E6B18B03C4AC3F41FF66E611119B5B41

The **pathway_edges_noded** table is created after executing the following SQL query. This command basically processes an existing **edges table** (e.g., **pathway_edges**) to introduce **nodes** at intersections or breakpoints. This transformation ensures that the pathway network properly represents connectivity at junctions.

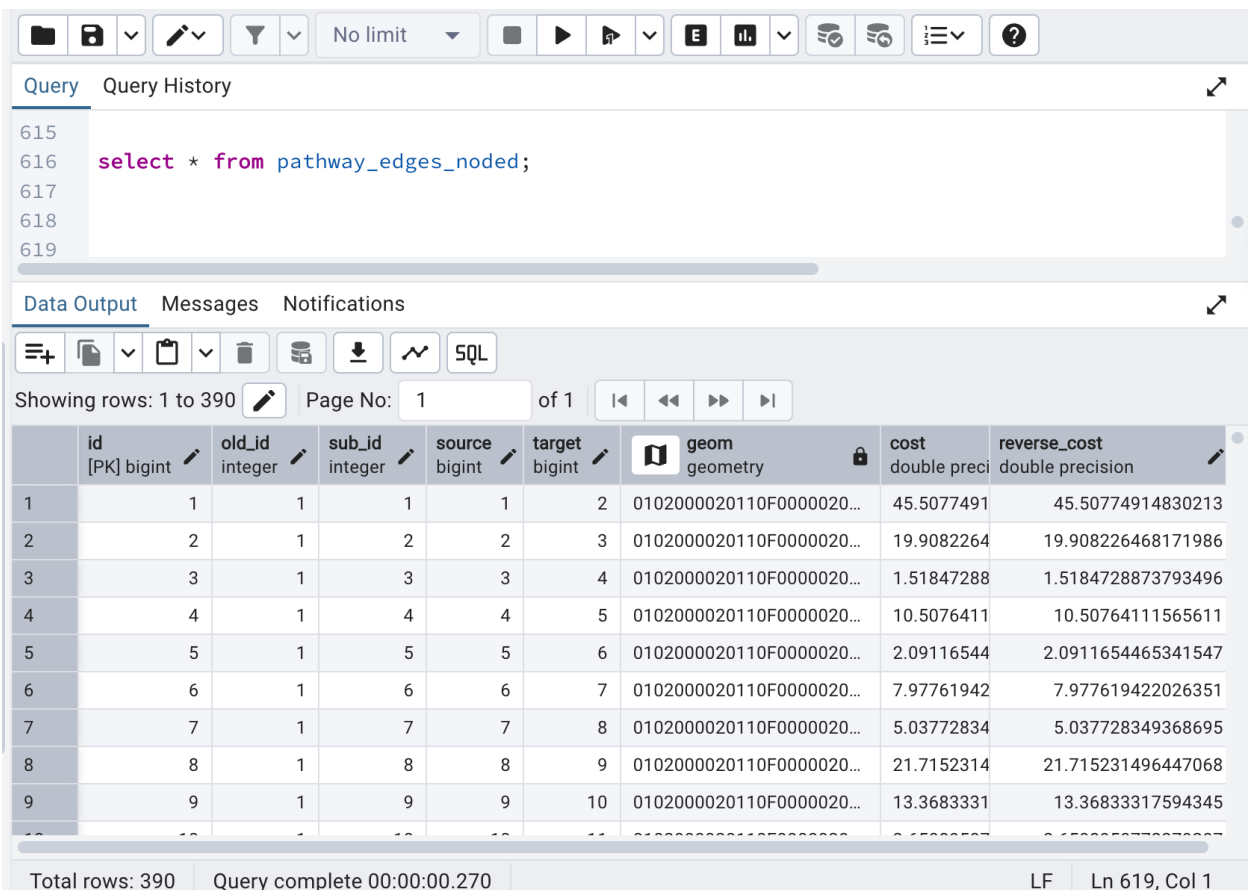
```
SELECT pgr_nodeNetwork('pathway_edges', 0.5, the_geom => 'geom');
```

Four new columns are added to the **pathway_edges_noded** table to store essential information for network analysis and routing. These columns typically include the source (starting node) and target (ending node) of each edge. After adding these columns, the **pgr_createTopology** SQL function is executed to populate the source and target columns based on the geometry of the pathways. It also creates a new table "**pathway_edges_noded_vertices_pgr**", which represents the network topology in a vertex-edge format. This table is important to calculate the shortest path or optimal routes in the network.

```
ALTER TABLE pathway_edges_noded ADD COLUMN source INTEGER;
ALTER TABLE pathway_edges_noded ADD COLUMN target INTEGER;
ALTER TABLE pathway_edges_noded ADD COLUMN cost DOUBLE PRECISION;
ALTER TABLE pathway_edges_noded ADD COLUMN reverse_cost DOUBLE PRECISION;
```

```
SELECT pgr_createTopology('pathway_edges_noded', 0.5, 'geom', 'id');
```

```
UPDATE pathway_edges_noded SET
cost = ST_length(geom),
reverse_cost = ST_length(geom);
```



The screenshot shows a database query interface. The top section displays the SQL commands executed, including the `SELECT pgr_nodeNetwork` and `ALTER TABLE` statements. Below the query editor, the "Data Output" tab is active, showing a table with 9 rows of data. The table has columns for `id`, `old_id`, `sub_id`, `source`, `target`, `geom`, `cost`, and `reverse_cost`. The data shows a sequence of nodes connected by edges, with the `geom` column containing geometry data and the `cost` and `reverse_cost` columns containing numerical values.

	id [PK] bigint	old_id integer	sub_id integer	source bigint	target bigint	geom geometry	cost double preci	reverse_cost double precision
1	1	1	1	1	2	0102000020110F00000020...	45.5077491	45.50774914830213
2	2	1	2	2	3	0102000020110F00000020...	19.9082264	19.908226468171986
3	3	1	3	3	4	0102000020110F00000020...	1.51847288	1.5184728873793496
4	4	1	4	4	5	0102000020110F00000020...	10.5076411	10.50764111565611
5	5	1	5	5	6	0102000020110F00000020...	2.09116544	2.0911654465341547
6	6	1	6	6	7	0102000020110F00000020...	7.97761942	7.977619422026351
7	7	1	7	7	8	0102000020110F00000020...	5.03772834	5.037728349368695
8	8	1	8	8	9	0102000020110F00000020...	21.7152314	21.715231496447068
9	9	1	9	9	10	0102000020110F00000020...	13.3683331	13.36833317594345

Total rows: 390 Query complete 00:00:00.270 LF Ln 619, Col 1

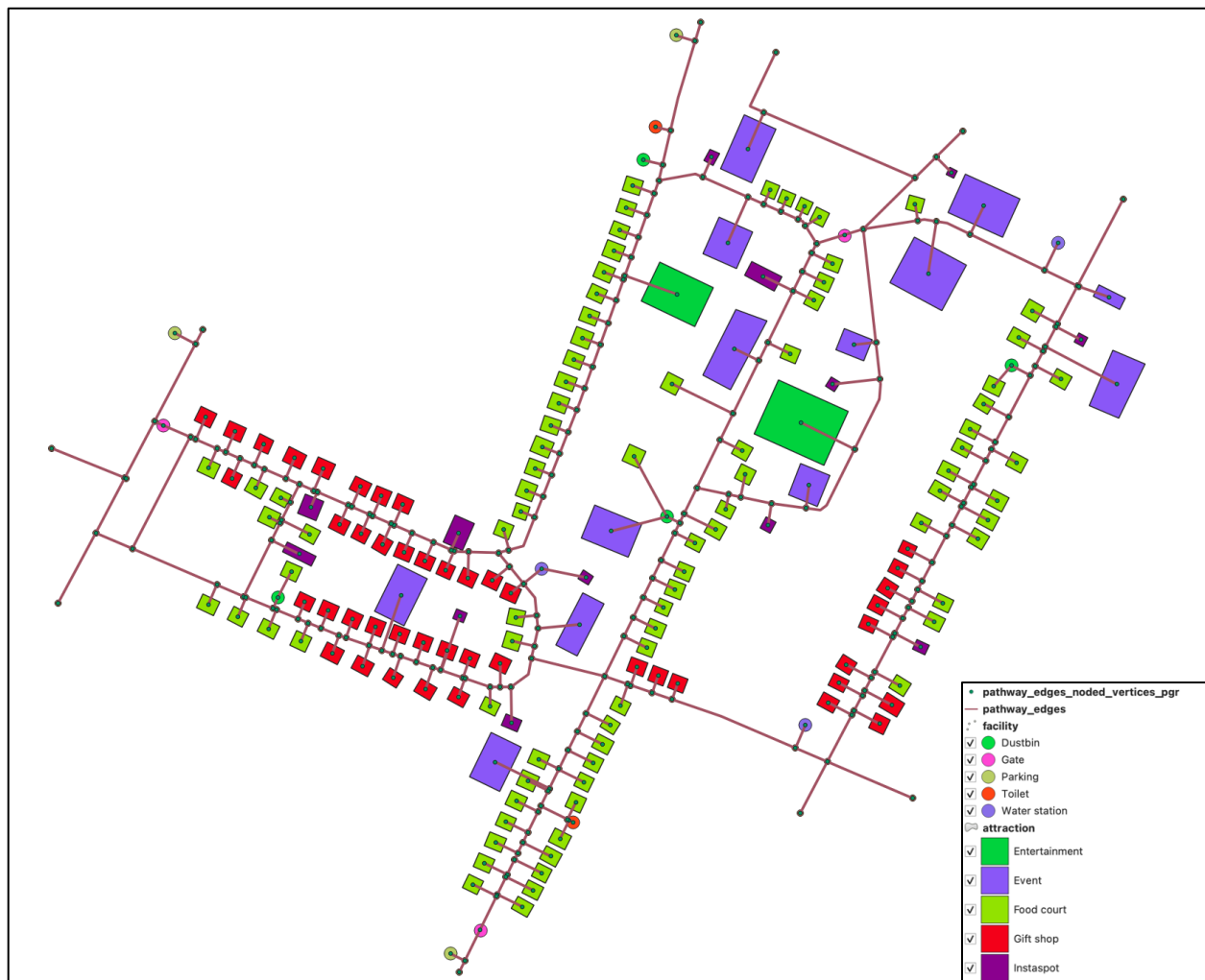


Figure 12: Network Topology Map

6.3 Routing Queries

a. Route nearest facility based on facility_type

[Pgr_dijkstra](#) algorithm was used to calculate the shortest path. It is a graph search algorithm that solves the shortest path problem for a graph with non-negative edge path costs, producing a shortest path from a starting vertex to an ending vertex. This implementation can be used with a directed graph and an undirected graph. The following PREPARE statement is defined to calculate and generate the shortest path to the nearest facility. The facility_type here is passed as float8, which is stored as a foreign key in the facility table.

```
PREPARE route_nearest_facility (float8, float8, float8) AS
WITH nearest_node AS (
  SELECT id
  FROM pathway_edges_noded_vertices_pgr
  ORDER BY ST_Distance(the_geom, ST_SetSRID(ST_MakePoint($1, $2), 3857))
  LIMIT 1
),
event_nodes AS (
  SELECT id, name,
    (SELECT id FROM pathway_edges_noded_vertices_pgr ORDER BY
  ST_Distance(the_geom, e.geom) LIMIT 1) AS node_id
  FROM facility e
  WHERE type_id=$3
),
paths AS (
  SELECT * FROM pgr_dijkstra(
    'SELECT id, source, target, cost FROM pathway_edges_noded',
    (SELECT id FROM nearest_node),
    ARRAY(SELECT node_id FROM event_nodes), -- Pass multiple event nodes as an array
    false
  )
),
path_cost AS (
  SELECT r.end_vid AS node, SUM(r.cost) AS total_cost
  FROM paths r
  GROUP BY r.end_vid
  ORDER BY r.end_vid
),
shortest_node AS (
  SELECT node AS id
  FROM path_cost
  ORDER BY total_cost
  LIMIT 1
),
-- select * from shortest_node;

route AS (
  SELECT * FROM pgr_dijkstra(
    'SELECT id, source, target, cost FROM pathway_edges_noded',
    (SELECT id FROM nearest_node),
    (SELECT id FROM shortest_node),
    false
  )
)

SELECT e.*
FROM pathway_edges_noded e
JOIN route r ON e.id = r.edge;

EXECUTE route_nearest_facility(2075924.4, 7236775.2, 2)
```

Info

Table

Preview

Query (city_festival_management) X

SOL

Saved query

Name

Save

Delete

Load File

Save As File

52

53

54 EXECUTE route_nearest_facility(2075924.4, 7236775.2, 2);

55

Execute

32 rows,0.013 seconds

Create a view

Clear

Query History

	id	old_id	sub_id	source	target	geom	cost	reverse_cost
1	425	178	1	331	169	0102000020...	10.68202751...	10.68202751...
2	183	12	6	169	170	0102000020...	5....	5....
3	184	12	7	170	140	0102000020...	15.02735816...	15.02735816...
4	147	8	10	139	140	0102000020...	18.18790083...	18.18790083...
5	146	8	8	138	139	0102000020...	5....	5....
6	145	8	7	137	138	0102000020...	14.31187923...	14.31187923...
7	144	8	6	136	137	0102000020...	9....	9....
8	143	8	5	135	136	0102000020...	9....	9....
9	142	8	4	134	135	0102000020...	10.48553501...	10.48553501...

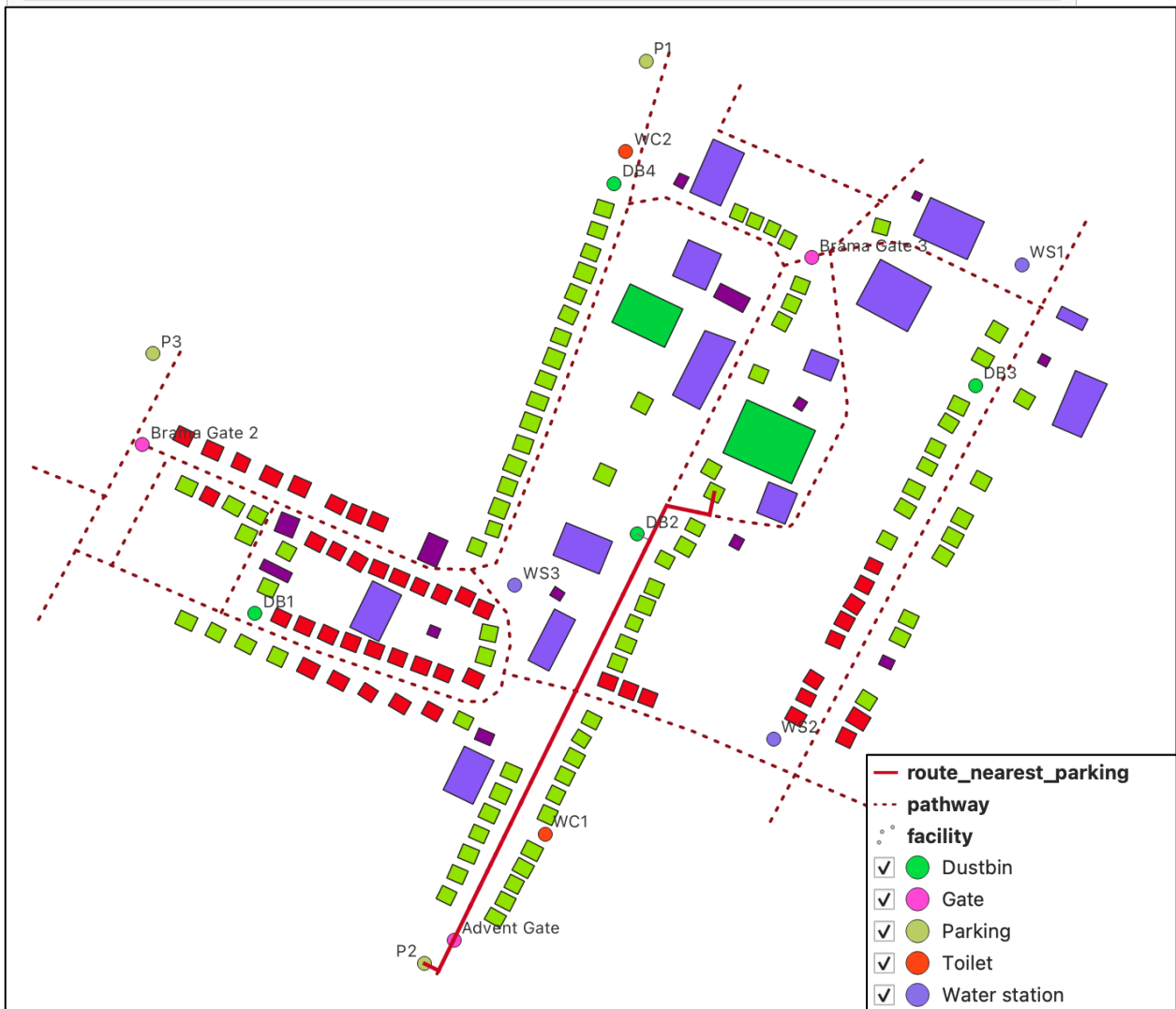


Figure 13: Route to the nearest parking facility

b. Route nearest attraction

Similarly, the above query can be updated to get the shortest path to the attractions.

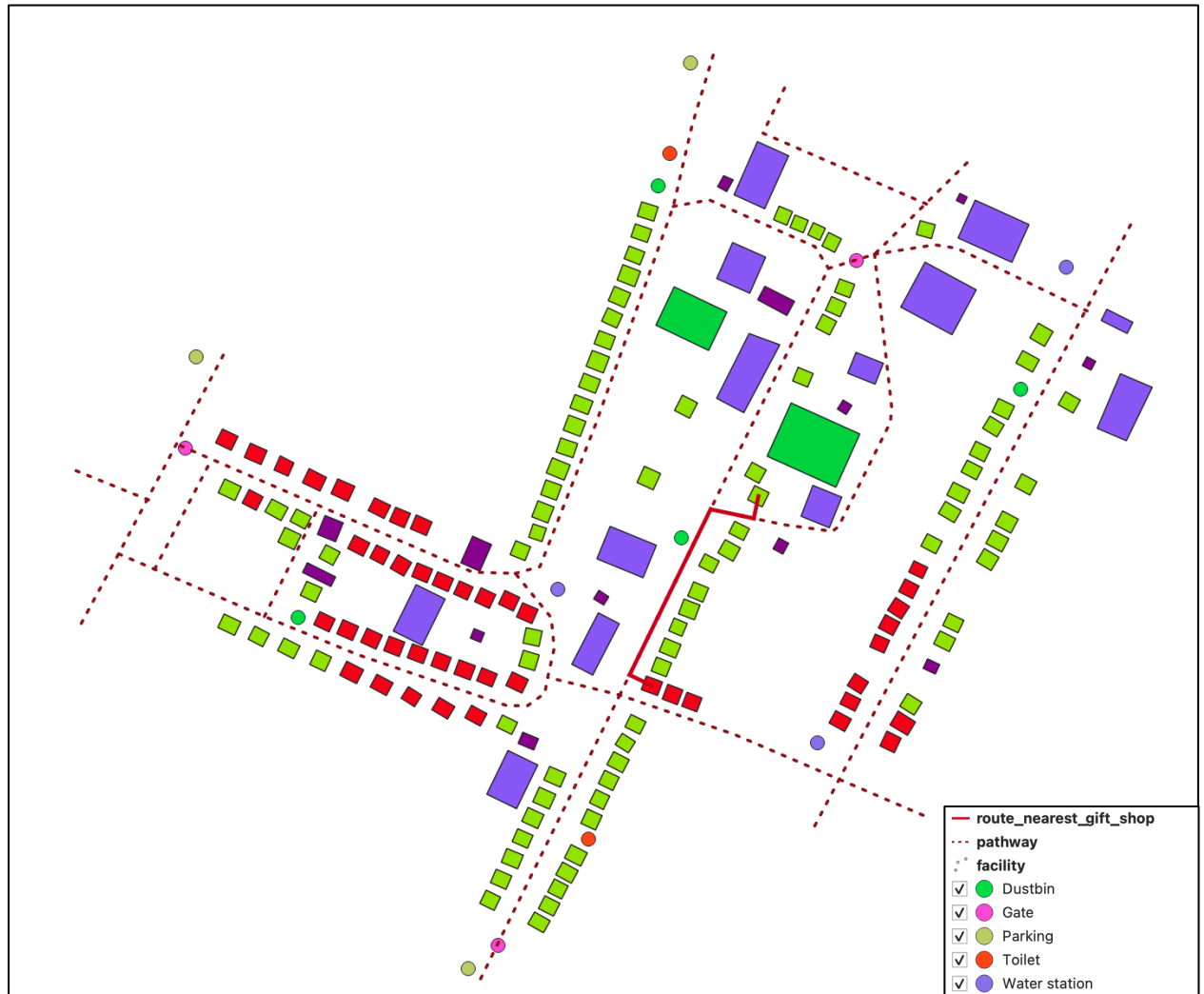


Figure 14: Route nearest gift shop

Roles and permissions need to be updated accordingly for the users to access these routing queries by the visitors, staff, and managers. In addition to this, as attraction or facility points might be added, changed, or removed, the tables related to routing need to be updated.

7 EXPORT DDL AND DML

The following `pg_dump` command is executed in a terminal to export DDL, DML, and both.

`-s` in command exports schemas (DDL) only, `-a` in command exports INSERT data (DML) only, `-d` exports both DDL and DML. By using these SQL, the database can be easily replicated.

```
pg_dump -U postgres -h localhost -p 5432 -s city_festival_management > DDL_city_festival_management.sql
```

```
pg_dump -U postgres -h localhost -p 5432 -a city_festival_management > DML_city_festival_management.sql
```

```
pg_dump -U postgres -h localhost -p 5432 -d city_festival_management > city_festival_management.sql
```

8 CONCLUSION

In this project, a spatial database was created and developed from scratch for the Gdańsk Christmas fair. With this, the broad concept of spatial data preparation, the importance of building the ER diagram, normalizing the table, denormalization, user roles and permission, and spatial queries based on user locations, non-spatial queries, and shortest path routing were fully exploited.

REFERENCES

The magic of Christmas in the center of Gdansk. Crowds at the Christmas Market. (2024, Dec 9). From TVN24: <https://tvn24.pl/trojmiasto/jarmark-bozonarodzeniowy-w-gdansku-2024-atrakcje-produkty-smakolyki-st8186405>